PERTEMUAN KE-1
Machine Learning

Pengenalan Machine Learning

1. Definisi Machine Learning (ML)

Machine Learning (ML) adalah cabang dari Artificial Intelligence (AI) yang memungkinkan sistem komputer belajar dari data tanpa diprogram secara eksplisit. ML menggunakan algoritma untuk mengidentifikasi pola dalam data, membuat prediksi, atau mengambil keputusan berdasarkan pengalaman (data historis).

Ciri Khas :

- Belajar dari Data: ML mengandalkan dataset untuk melatih model.
- Meningkatkan Performa: Semakin banyak data, semakin akurat model.
- Otomatisasi: Tidak memerlukan instruksi manual untuk setiap kasus.

Contoh Penggunaan :

- Rekomendasi produk (Amazon, Netflix).
- Prediksi harga saham.
- Deteksi spam email.

2. Perbedaan AI, ML, dan DL

a. Artificial Intelligence (AI)

Definisi: AI adalah bidang luas yang mencakup segala sistem yang dapat meniru kecerdasan manusia (misalnya: penalaran, pemecahan masalah, pembelajaran).

Cakupan :

- AI mencakup ML, DL, rule-based systems, robotics, NLP, dll.
- Tidak semua AI menggunakan pembelajaran berbasis data (contoh: sistem berbasis aturan/rule-based AI) ex : *chatgpt*.

b. Machine Learning (ML)

Definisi: Subset AI yang fokus pada pembuatan model belajar dari data

Teknik Umum :

- Supervised Learning (contoh: klasifikasi, regresi)
- Unsupervised Learning (contoh: clustering).
- Reinforcement Learning (contoh: AlphaGo).

Contoh ML :

- Prediksi menggunakan regresi linear
- Klasifikasi dengan menggunakan decision tree

c. Deep Learning (DL)

Definisi: Subset ML yang menggunakan jaringan saraf tiruan (neural networks) dengan banyak lapisan (*deep*).

Ciri Khas:

- Membutuhkan data besar dan komputasi intensif.
- Dapat mengekstrak fitur secara otomatis (tanpa feature engineering manual).

Contoh Aplikasi DL:

- Pengenalan gambar (CNN).
- Terjemahan bahasa (RNN, Transformer).

- Mobil otonom.
- Pengenalan Wajah

3. Tabel Perbandingan

Aspek	AI	ML	DL
Definisi	Sistem cerdas secara umum	Belajar dari data	ML dengan neural networks deep
Ketergantungan Data	Tidak selalu	Ya	Ya (dalam jumlah besar)
Kompleksitas	Variatif	Menengah	Sangat tinggi
Contoh Algoritma	Rule-based systems	Linear Regression, Decision Tree	CNN, RNN, Transformer

4. Supervised Learning

Supervised Learning (Pembelajaran Terawasi) adalah salah satu pendekatan dalam Machine Learning di mana model dilatih menggunakan dataset yang sudah berlabel (labeled data). Artinya, setiap input data (fitur) memiliki output yang benar (label/target) yang sudah diketahui. Tujuan model adalah mempelajari hubungan antara fitur dan label sehingga dapat membuat prediksi akurat pada data baru yang belum pernah dilihat.

Contoh Sederhana :

- Input Fitur : Ukuran rumah, jumlah kamar, lokasi.
- Output Label : Harga rumah.
- Tugas Model : Memprediksi harga rumah baru berdasarkan fitur yang diberikan.

Konsep Dasar

a. Dataset

- Fitur (X): Variabel independen (misalnya: usia, pendapatan).
- Label (Y): Variabel dependen/target (misalnya: risiko kredit, jenis penyakit).

b. Proses Pembelajaran (Trining Model)

- Training Phase: Model belajar dari data latih (fitur + label).
- Testing Phase: Model diuji dengan data baru tanpa label untuk mengukur akurasi.

c. Tipe Masalah

Tujuan: Memetakan input ke dalam kategori diskrit.

1. Klasifikasi

   Tujuan: Memetakan input ke dalam kategori diskrit.

   Contoh :

   - Deteksi spam (spam vs. bukan spam).
   - Diagnosa penyakit (positif vs. negatif).

   Algoritma Umum :

   - Logistic Regression
   - Decision Trees
   - Support Vector Machines (SVM)
   - Neural Networks

2. Regresi

   Tujuan: Memprediksi nilai kontinu.

Contoh :

- Prediksi harga saham
- Estimasi umur baterai handphone

Algoritma Umum:

- Linear Regression
- Polynomial Regression
- Random Forest Regression

d. Algoritma Populer

Algoritma	Tipe Masalah	Penjelasan Singkat
Linear Regression	Regresi	Memodelkan hubungan linear antara fitur dan label.
Logistic Regression	Klasifikasi	Memprediksi probabilitas kelas biner (0/1).
Decision Trees	Klasifikasi & Regresi	Membuat struktur pohon keputusan berdasarkan aturan if-else.
Random Forest	Klasifikasi & Regresi	Gabungan banyak Decision Trees untuk meningkatkan akurasi.
SVM	Klasifikasi	Mencari hyperplane terbaik untuk memisahkan kelas.
K-Nearest Neighbors (KNN)	Klasifikasi & Regresi	Prediksi berdasarkan kedekatan dengan tetangga terdekat.

e. Langkah-Langkah Implementasi

1. Pengumpulan Data

   Contoh: Dataset rumah (fitur: luas, lokasi; label: harga).

2. Preprocessing Data

   Handling missing values, normalisasi, encoding kategorikal.

3. Pemilihan Algoritma

   Pilih model sesuai masalah (klasifikasi/regresi).

4. Trining Model

   Pisahkan data menjadi training set (70-80%) dan test set (20-30%).

5. Evaluasi Model

   - Metrik untuk Klasifikasi: Akurasi, Precision, Recall, F1-Score.
   - Metrik untuk Regresi: MSE (Mean Squared Error), R^2 Score.

f. Contoh Kasus

1. Prediksi Spam Email

```python
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Data contoh: teks email dan label (0 = bukan spam, 1 = spam)
emails = ["Diskon besar-besaran!", "Meeting besok jam 10", "Menang undian Rp 1 juta"]
labels = [1, 0, 1]

# Ekstraksi fitur (ubah teks ke vektor)
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)

# Bagi data training & testing
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2)

# Latih model Naive Bayes
model = MultinomialNB()
model.fit(X_train, y_train)

# Prediksi
y_pred = model.predict(X_test)
print("Akurasi:", accuracy_score(y_test, y_pred))
```

2. Prediksi Harga Rumah

```python
from sklearn.linear_model import LinearRegression
import numpy as np

# Data contoh: luas rumah (m²) vs harga (juta Rp)
X = np.array([[50], [70], [90]])  # Fitur
y = np.array([300, 420, 500])     # Label

# Latih model regresi linear
model = LinearRegression()
model.fit(X, y)

# Prediksi harga rumah 80 m²
prediksi = model.predict(np.array([[80]]))
print(f"Prediksi harga: Rp {prediksi[0]:.0f} juta")
```

g. Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Akurat untuk data terlabel.	Membutuhkan data berlabel yang banyak.
Mudah diinterpretasikan (terutama model linear).	Rentan overfitting jika data noisy.
Bisa digunakan untuk klasifikasi & regresi.	Performa buruk jika pola data kompleks (perlu DL).

5. Unsupervised Learning

Unsupervised Learning (Pembelajaran Tidak Terawasi) adalah pendekatan dalam Machine Learning di mana model belajar dari data tanpa label (unlabeled data). Tujuannya adalah menemukan pola, struktur, atau hubungan tersembunyi dalam data tanpa panduan output yang benar.

Contoh Sederhana:

- Input: Data pembelian pelanggan (tanpa kategori).
- Tugas Model: Mengelompokkan pelanggan berdasarkan perilaku belanja.

Konsep Dasar

a. Data (Tanpa Label)
   - Tidak ada variabel target (label), hanya fitur (X).
   - Model berusaha memahami struktur data secara mandiri.

b. Tipe Utama
   1. Clustering (Pengelompokan)
      - Membagi data ke dalam kelompok (cluster) yang memiliki kemiripan.
      - Contoh: Segmentasi pelanggan, pengelompokan dokumen.
   2. Dimensionality Reduction (Reduksi Dimensi)
      - Mengurangi jumlah fitur dengan mempertahankan informasi penting.
      - Contoh: Visualisasi data high-dimensional, kompresi gambar.
   3. Association Rule Learning (Aturan Asosiasi)
      - Menemukan hubungan antar-item dalam dataset.
      - Contoh: Market basket analysis (rekomendasi produk).

c. Algoritma Populer

Algoritma	Tipe Masalah	Penjelasan Singkat
K-Means Clustering	Clustering	Membagi data ke dalam K cluster berbasis jarak.
Hierarchical Clustering	Clustering	Membuat dendrogram untuk pengelompokan hierarkis.
DBSCAN	Clustering	Clustering berbasis kepadatan (density-based).
PCA (Principal Component Analysis)	Dimensionality Reduction	Mengubah fitur ke dimensi yang lebih rendah dengan variansi maksimal.
t-SNE	Dimensionality Reduction	Reduksi dimensi untuk visualisasi data kompleks.
Apriori	Association Rule	Menemukan pola "jika A maka B" dalam transaksi.

d. Langkah-Langkah Implementasi

1. Pengumpulan Data
   - Contoh: Data transaksi pelanggan, gambar tanpa label.
2. Preprocessing Data
   - Normalisasi, handling missing values, encoding kategorikal.
3. Pemilihan Algoritma
   - Pilih berdasarkan tujuan (clustering, reduksi dimensi, dll.).
4. Pelatihan Model
   - Model belajar pola dari data tanpa label.
5. Evaluasi
   - Clustering: Metrik seperti Silhouette Score, Elbow Method.
   - Dimensionality Reduction: Preservasi variansi (PCA).

e. Contoh Kasus

1. Customer Clustering

```python
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import numpy as np

# Data contoh: penghasilan vs pengeluaran pelanggan (dalam juta Rp)
X = np.array([
    [3, 2], [4, 3], [10, 8],
    [15, 12], [20, 18], [30, 25]
])

# Gunakan K-Means dengan 2 cluster
kmeans = KMeans(n_clusters=2)
kmeans.fit(X)
labels = kmeans.predict(X)

# Visualisasi
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
plt.xlabel("Penghasilan")
plt.ylabel("Pengeluaran")
plt.title("Clustering Pelanggan")
plt.show()
```

2. PCA Iris Dataset

```python
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris

# Load dataset Iris (4 fitur -> direduksi jadi 2D)
data = load_iris()
X = data.data

# Reduksi dimensi dengan PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Visualisasi
plt.scatter(X_pca[:,0], X_pca[:,1], c=data.target)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA pada Iris Dataset")
plt.show()
```

f. Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Tidak membutuhkan data berlabel.	Hasil sulit dievaluasi (tidak ada ground truth).
Dapat menemukan pola tak terduga.	Membutuhkan tuning parameter (misalnya: jumlah cluster).
Cocok untuk eksplorasi data.	Interpretasi cluster bisa subjektif.

6. Reinforcement Learning

Reinforcement Learning (RL) adalah paradigma Machine Learning di mana sebuah agen (agent) belajar untuk mengambil tindakan (actions) dalam suatu lingkungan (environment) untuk memaksimalkan reward (imbalan) kumulatif. Berbeda dengan Supervised dan Unsupervised Learning, RL tidak bergantung pada data berlabel melainkan pada interaksi dinamis antara agen dan lingkungan.

Ciri-ciri

- Belajar dari Pengalaman: Agen belajar melalui trial and error.
- Reward-based: Tujuan utama adalah memaksimalkan reward.
- Sequential Decision Making: Keputusan sekarang memengaruhi keadaan di masa depan.

Contoh :

- Robot belajar berjalan.
- AI bermain game (AlphaGo, Mobile Legend AFK).
- Manajemen portofolio otomatis di trading.

a. Komponen Utama

Komponen	Penjelasan	Contoh
Agent	Entitas yang belajar dan mengambil keputusan.	AI dalam game, robot.
Environment	Dunia tempat agen berinteraksi.	Papan catur, jalanan untuk mobil otonom.
State (s)	Situasi saat ini di lingkungan.	Posisi robot, skor game.
Action (a)	Tindakan yang diambil agen.	Bergerak, membeli/menjual saham.
Reward (r)	Feedback numerik setelah tindakan (bisa positif/negatif).	+1 untuk menang, -1 untuk kalah.
Policy (π)	Strategi yang menentukan tindakan berdasarkan state.	"Jika di state A, lakukan action X."
Value Function (V)	Estimasi reward jangka panjang dari suatu state.	"State ini bernilai +5 dalam jangka panjang."
Q-Function (Q)	Nilai expected reward dari pasangan (state, action).	"Di state S, action A menghasilkan Q=10."

b. Proses Pembelajaran

- Agen mengamati state (S_t) dari environment.
- Agen memilih action (A_t) berdasarkan policy (π).
- Environment memberikan reward (R_{t+1}) dan state baru (S_{t+1}).
- Agen memperbarui policy atau value function berdasarkan feedback.
- Proses berulang hingga mencapai tujuan (convergence).

Contoh alur :

- State: Posisi karakter dalam game.
- Action: Bergerak ke kiri/kanan.

- Reward: +10 jika menang, -5 jika kalah.
- Policy Update: Belajar dari kemenangan/kekalahan.

c. Kategori Algoritma

a. Model-Based vs Model-Free RL

Tipe	Penjelasan	Contoh Algoritma
Model-Based	Agen memahami model lingkungan (transition & reward function).	Dynamic Programming (Value Iteration).
Model-Free	Agen tidak tahu model lingkungan, belajar dari pengalaman.	Q-Learning, Deep Q-Network (DQN).

b. Value-Based vs Policy-Based vs Actor-Critic

Tipe	Penjelasan	Contoh Algoritma
Value-Based	Fokus pada estimasi value function (Q/V).	Q-Learning, SARSA.
Policy-Based	Langsung mengoptimalkan policy (tanpa value function).	REINFORCE.
Actor-Critic	Gabungan value-based dan policy-based.	A3C, PPO.

d. Algoritma

Algoritma	Kategori	Penjelasan Singkat
Q-Learning	Model-Free, Value-Based	Belajar Q-values dari pengalaman.
Deep Q-Network (DQN)	Model-Free, Value-Based	Q-Learning + Deep Neural Networks.
Policy Gradients (REINFORCE)	Policy-Based	Optimasi policy secara langsung.
Proximal Policy Optimization (PPO)	Actor-Critic	Stabil dan efisien untuk RL kompleks.
Monte Carlo Methods	Model-Free	Belajar dari episode lengkap.

e. Contoh Implementasi

1. Q-Learning untuk Maze Solver

import numpy as np

```python
import numpy as np

# Definisi environment (maze 3x3)
# 0 = jalan, 1 = tembok, 2 = goal
maze = np.array([
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 2]
])

# Inisialisasi Q-table (states x actions)
q_table = np.zeros((9, 4))  # 9 states (3x3), 4 actions (up, down, left, right)

# Hyperparameters
alpha = 0.1  # Learning rate
gamma = 0.9  # Discount factor
epsilon = 0.1  # Exploration rate

# Training loop
for episode in range(1000):
    state = 0  # Start at (0,0)
    done = False

    while not done:
        # Epsilon-greedy action selection
        if np.random.uniform(0, 1) < epsilon:
            action = np.random.randint(0, 4)
        else:
            action = np.argmax(q_table[state])

        # Execute action
        next_state, reward, done = take_action(state, action, maze)

        # Update Q-table
        q_table[state, action] += alpha * (reward + gamma * np.max(q_table[
        next_state]) - q_table[state, action])
        state = next_state
```

2. Deep Q-Learning (DQN) dengan TensorFlow

import tensorflow as tf

```python
import tensorflow as tf
from tensorflow.keras import layers

# Bangun Deep Q-Network (DQN)
model = tf.keras.Sequential([
    layers.Dense(24, activation='relu', input_shape=(4,)),
    layers.Dense(24, activation='relu'),
    layers.Dense(2, activation='linear')  # Output: Q-values untuk 2 actions
])

# Loss function (Huber loss) dan optimizer
model.compile(optimizer='adam', loss=tf.keras.losses.Huber())
```

f. Tantangan

Exploration vs Exploitation

a. Harus menyeimbangkan mencoba hal baru (exploration) dan memanfaatkan pengetahuan (exploitation).

Reward Sparse

b. Reward jarang (misal: hanya di akhir game) membuat pembelajaran lambat.

High-Dimensional State Space

c. Jika state terlalu kompleks (misal: gambar), dibutuhkan Deep RL.

Referensi:

- *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (O'Reilly).*